UNIVERSITÉ D'ANTSIRANANA

ÉCOLE NORMALE SUPÉRIEURE POUR L'ENSEIGNEMENT TECHNIQUE

Domaine : Sciences de l'éducation

Mention : Éducation-Apprentissage-Didactique et Ingénierie en Mathématiques et Informatique

Parcours : Professorat d'Enseignement en Génie Informatique

PROJET

Conception d'un éditeur collaboratif de code base sur des CRDT optimisés pour la synchronisation en temps réel dans des environnements distribués

Pour l'obtention du diplôme

Master d'Aptitude au Professorat de l'Ecole Normale (MAPEN)

Réalisé par : SAVATRA Houzefa

Soutenu le 00 mars 2026

Devant les membres des jury :

Président	: *
Encadreur	: Docteur RALAHADY Bruno Bakys
Examinateur	: *

Année Universitaire : 2024-2025

UNIVERSITÉ D'ANTSIRANANA

ÉCOLE NORMALE SUPÉRIEURE POUR L'ENSEIGNEMENT TECHNIQUE

Domaine : Sciences de l'éducation

Mention : Éducation-Apprentissage-Didactique et Ingénierie en Mathématiques et Informatique

Parcours : Professorat d'Enseignement en Génie Informatique

PROJET

Conception d'un éditeur collaboratif de code base sur des CRDT optimisés pour la synchronisation en temps réel dans des environnements distribués

Pour l'obtention du diplôme

Master d'Aptitude au Professorat de l'Ecole Normale (MAPEN)

Réalisé par : SAVATRA Houzefa

Soutenu le 00 mars 2026

Devant les membres des jury :

Président	: *
Encadreur	: Docteur RALAHADY Bruno Bakys
Examinateur	: *

Année Universitaire : 2024-2025

Remerciements

Tout d'abord, nous tenons à remercier l'Ecole Normale Supérieure pour l'Enseignement Technique avec son Directeur, Docteur ULRICH Canisius.

Ensuite, nous transmettrons nos sincères remerciements à Professeur RALAHADY Buno Bakys qui est le responsable du mention EADIMI et aussi mon encadreur pédagogique qui a suivi et orienté notre travail avec patience et pertinence. Merci pour le temps que vous m'avez accordé pour la correction, ainsi que les conseils qui ont permis la réalisation de ce mémoire.

Et aussi nous remercions, Monsieur RAVELOMANANA Juvenile Christalin, le responsable du parcours Professorat l'Enseignement en Génie Informatique niveau Licence, qui a accepté d'être mon examinateur.

Après cela, nous adressons une immense gratitude à Docteur TOVOHERY Josoa, le responsable du parcours Professorat l'Enseignement en Génie Informatique niveau Master.

Nous remercions respectueusement Docteur RAKOTOMALALA Hery Frédéric, président du jury, pour votre bienveillance et l'attention que vous avez accordée à mon travail.

En outre, nos sincères reconnaissance reviennent à :

- Tous les enseignants de l'ENSET ainsi que le personnel administratif sans votre appui pédagogique et technique, ce mémoire n'aurait pas pu être réalisé.
- Tous les professionnels qui nous ont accordé leurs temps et leur confiance pendant les entretiens, sans eux nous n'aurons pas pu avoir des Données pour réaliser ce présent mémoire

Nous tenons à remercier particulièrement tous les membres de ma famille pour leur soutien moral, financier et matériel et surtout pour l'accomplissement de ce mémoire du début jusqu'à la fin.
Enfin, il nous a paru primordial, de remercier tous ceux qui avaient apporte leurs aides lors de la réalisation de ce mémoire.
Merci beaucoup !

Cahier des charges

Thème du sujet :

⇒ Conception d'un éditeur collaboratif de code base sur des CRDT optimisés pour la synchronisation en temps réel dans des environnements distribués

Contextes

Le développement logiciel collaboratif à distance nécessite des outils capables de gérer simultanément les modifications de plusieurs utilisateurs sans conflits ni perte de données. Les IDE classiques et systèmes de version centralisés montrent leurs limites pour la synchronisation en temps réel. Les CRDT (Conflict-Free Replicated Data Types) offrent une solution efficace pour garantir la cohérence automatique dans les environnements distribués. Ce projet vise à concevoir un éditeur de code collaboratif utilisant des CRDT optimisés, assurant synchronisation instantanée, portabilité multiplateforme et interface ergonomique, tout en intégrant les fonctionnalités avancées attendues d'un IDE moderne.

Objectifs généraux

Concevoir un éditeur de code multiplateforme pour Java, Python et HTML, avec synchronisation en temps réel, interface ergonomique et stable, supportant la collaboration distribuée et le partage de fichiers.

Objectifs spécifiques

- Gérer automatiquement les projets et fichiers
- Proposer une interface moderne avec coloration syntaxique enrichie et support des multiples onglets
- Assurer la synchronisation collaborative des modifications en temps réel
- Permettre l'exécution directe des fichiers selon leur type (Java, Python, HTML)
- Inclure des tests intégrés pour la stabilité et la performance
- Fournir un packaging multiplateforme pour Linux, Windows et macOS

Contraintes

Performance même sur projets volumineux, stabilité et prévention de perte de données, compatibilité multiplateforme, sécurité lors du partage, architecture modulaire pour faciliter l'extension future.

Table des matières

Remerciements i

Cahier des Charges ii

Liste des tableaux vi

Liste des figures vii

1 Liste des symboles viii

Liste des symboles viii

Introduction Générale 1

I Fondements Théoriques 2

1 Systèmes Distribués 3
- 1.1 Définition . . . 3
- 1.2 Cohérence et réplication . . . 3
- 1.3 Théorème CAP . . . 3

2 Synchronisation Collaborative 4
- 2.1 Operational Transformation . . . 4
- 2.2 CRDT . . . 4
- 2.3 Comparaison OT vs CRDT . . . 4
- 2.4 Justification du choix . . . 5

3 État de l'Art des IDE Modernes 7
- 3.1 Analyse comparative . . . 7
- 3.2 Limites des solutions existantes . . . 7

II Conception et Architecture du Système 8

1 Analyse et Spécification des Besoins 9
- 1.1 **Exigences fonctionnelles** . . . 9
- 1.2 **Exigences non fonctionnelles** . . . 9
- 1.3 **Contraintes** . . . 10

2 Architecture Générale du Système **11**
2.1 **Architecture logicielle modulaire** . . . 11
2.2 **Architecture réseau** . . . 12
2.3 **Modélisation UML** . . . 12

3 Conception Technique Détaillée **13**
3.1 **Moteur CRDT** . . . 13
3.2 **Serveur WebSocket** . . . 13
3.3 **Interface JavaFX** . . . 14
3.4 **Gestionnaire de projets** . . . 15
3.5 **Système de plugins** . . . 15

III Implémentation et Validation **16**

4 Implémentation **17**
4.1 Technologies utilisées . . . 17
4.2 Structure du projet . . . 17
4.3 Extraits de code . . . 17

5 Tests et Validation **18**
5.1 Tests unitaires . . . 18
5.2 Tests d'intégration . . . 18
5.3 Tests multi-clients . . . 18
5.4 Tests de performance . . . 18

6 Résultats et Analyse **19**
6.1 Performance . . . 19
6.2 Robustesse . . . 19
6.3 Comparaison avec IDE existants . . . 19

7 Coût et Perspectives **20**
7.1 Estimation du coût . . . 20
7.2 Limites . . . 20
7.3 Améliorations futures . . . 20

8 Conclusion Générale **21**

Références bibliographiques et webographiques **I**

Annexe **II**

Liste des tableaux

2.1 Comparaison technique et formelle entre OT et CRDT pour un IDE collaboratif 5

Liste des figures

2.1 Architecture logicielle modulaire de l'IDE collaboratif avec ses couches et modules principaux 12

3.1 Synchronisation et diffusion des opérations CRDT 13
3.2 Distribution des opérations 14

Chapitre 1

Liste des symboles

SVT	Science de la Vie et de la Terre
TBI	Tableau Blancs Interactifs
WWW	World Wide Web
VS	Versus
TIC	Technologies de l'Information et de la Communication
USB	Universal Serial Bus

Introduction Générale

L'évolution rapide des environnements de développement logiciel a engendré une demande croissante pour des outils collaboratifs performants permettant à plusieurs développeurs de travailler simultanément sur un même projet, indépendamment de leur localisation géographique. Dans ce contexte, les systèmes distribués et les mécanismes de synchronisation en temps réel constituent des éléments fondamentaux pour garantir la cohérence des données, la continuité des modifications et l'amélioration de la productivité collective.

Face aux limites des éditeurs traditionnels dans les scénarios collaboratifs, la conception d'un éditeur de code basé sur des CRDT (Conflict-free Replicated Data Types) optimisés apparaît comme une solution pertinente pour gérer efficacement les modifications concurrentes sans nécessiter de verrouillage global. Cette approche permet d'assurer une résolution automatique des conflits tout en maintenant une expérience utilisateur fluide, réactive et cohérente.

L'objectif principal de ce projet est de concevoir et de développer un environnement de développement intégré (IDE) collaboratif capable de gérer la collaboration multi-utilisateur, la synchronisation en temps réel et la distribution efficace des modifications dans un environnement distribué. L'outil proposé vise également à offrir une interface ergonomique, moderne et stable, adaptée aux exigences actuelles du développement logiciel.

Ce mémoire présente, dans un premier temps, les fondements théoriques relatifs aux systèmes distribués et à la synchronisation collaborative. Il expose ensuite l'état de l'art des environnements de développement modernes, avant de détailler la conception et l'architecture technique de la solution proposée. Enfin, il décrit l'implémentation réalisée, les tests effectués, l'analyse des résultats obtenus ainsi que les perspectives d'amélioration, offrant ainsi une vision scientifique et complète de la démarche de conception et de développement adoptée.

Première partie

Fondements Théoriques

Chapitre 1

Systèmes Distribués

1.1 Définition

Un système distribué est un ensemble d'ordinateurs autonomes interconnectés par un réseau et coopérant pour atteindre un objectif commun. Ces machines communiquent et coordonnent leurs actions par échange de messages, tout en apparaissant à l'utilisateur comme un système unique et cohérent. Les systèmes distribués se caractérisent par l'absence d'horloge globale, la concurrence des processus, et la possibilité de pannes partielles.

1.2 Cohérence et réplication

La réplication des données constitue un mécanisme essentiel dans les systèmes distribués afin d'améliorer la disponibilité et la tolérance aux pannes. Elle consiste à maintenir plusieurs copies d'une même donnée sur différents nœuds du réseau.

La cohérence définit le degré de synchronisation entre ces copies. Plusieurs modèles de cohérence existent, notamment la cohérence forte, où toutes les répliques présentent un état identique à tout instant, et la cohérence éventuelle, où les répliques convergent progressivement vers un état commun après propagation des mises à jour.

1.3 Théorème CAP

Le théorème CAP, formulé par Eric Brewer, stipule qu'un système distribué ne peut garantir simultanément trois propriétés fondamentales :

- **Cohérence (Consistency)** : toutes les répliques retournent la même valeur.
- **Disponibilité (Availability)** : chaque requête reçoit une réponse.
- **Tolérance au partitionnement (Partition Tolerance)** : le système continue de fonctionner malgré une rupture de communication entre nœuds.

Dans un contexte de collaboration temps réel, le choix architectural privilégie généralement la disponibilité et la tolérance au partitionnement, au détriment d'une cohérence forte immédiate.

Chapitre 2

Synchronisation Collaborative

2.1 Operational Transformation

L'Operational Transformation (OT) est une technique de synchronisation permettant de gérer les modifications concurrentes sur un document partagé. Elle repose sur la transformation des opérations entrantes afin de préserver l'intention des utilisateurs malgré les conflits potentiels. Bien que performante, l'OT nécessite un contrôle centralisé et des mécanismes complexes de transformation, ce qui peut compliquer l'implémentation dans des environnements hautement distribués.

2.2 CRDT

Les CRDT (Conflict-free Replicated Data Types) sont des structures de données conçues pour garantir la convergence automatique des répliques sans coordination centrale stricte. Ils reposent sur des propriétés mathématiques telles que la commutativité et l'idempotence des opérations. Dans le cas des CRDT orientés opérations, chaque modification est propagée aux autres nœuds et appliquée localement de manière déterministe, garantissant ainsi la cohérence éventuelle.

2.3 Comparaison OT vs CRDT

L'**Operational Transformation (OT)** repose sur une logique de transformation dépendante du contexte et nécessite une gestion rigoureuse de l'ordre des opérations pour assurer la cohérence des documents partagés :

$$o_i' = T(o_i, o_j) \quad \text{où } T \text{ transforme } o_i \text{ en fonction de } o_j \text{ pour préserver l'intention.}$$

Les **CRDT (Conflict-free Replicated Data Types)** garantissent la convergence par construction. Chaque opération est commutative et idempotente, ce qui simplifie la gestion des conflits :

$$D_i \xrightarrow{op} D_i', \quad D_j \xrightarrow{op} D_j', \quad D_i' = D_j' \quad \forall i, j$$

Critère	**Operational Transformation (OT)**	**CRDT**
Principe	Transformation des opérations concurrentes : $o_i' = T(o_i, o_j)$	Réplication d'une structure de données avec opérations commutatives et idempotentes : $D \xrightarrow{op} D'$
Gestion des conflits	Fonctions de transformation explicites	Résolution automatique via commutativité et associativité
Modèle réseau	Serveur central pour ordonnancer	Pair-à-pair ou client-serveur sans coordination centrale
Complexité algorithmique	$O(n^2)$ pour documents longs	$O(\log n)$ ou $O(n)$ selon la structure choisie (RGA, LSEQ, Logoot)
Tolérance aux partitions	Sensible aux pertes et retards	Tolérant aux partitions, convergence assurée après réconciliation
Ordonnancement	Ordre causal strict nécessaire	L'ordre d'application n'affecte pas la convergence
Méta-données	Historique des opérations	Identifiant unique par élément (siteID + compteur), peut croître avec le document
Scalabilité	Limitée par le serveur central	Scalabilité horizontale naturelle
Exemples d'usage	Google Docs (OT historique)	Yjs, Automerge, Redis CRDT
Avantages IDE collaboratif	Gestion fine des conflits, historique complet	Convergence automatique, résilience réseau, idéal pour multi-utilisateurs
Inconvénients IDE	Complexité d'implémentation, risque d'erreurs	Surcharge mémoire due aux identifiants et métadonnées

Tableau 2.1 – Comparaison technique et formelle entre OT et CRDT pour un IDE collaboratif

Remarque : Bien que les CRDT simplifient la gestion des conflits et assurent une convergence déterministe, ils peuvent entraîner une augmentation de la consommation mémoire liée aux métadonnées nécessaires au suivi des opérations et à l'identification unique des éléments dans le document.

2.4 Justification du choix

Dans le cadre de ce projet, l'utilisation des CRDT est particulièrement justifiée par leur capacité à assurer une cohérence éventuelle entre plusieurs réplicas distribués sans nécessiter de serveur central pour la résolution des conflits. Chaque opération appliquée sur une

réplique est conçue pour être **commutative**, **idempotente** et **associative**, garantissant ainsi que, quel que soit l'ordre de propagation et d'application des modifications, toutes les copies convergeront vers un état final cohérent.

Cette approche présente plusieurs avantages majeurs pour un IDE collaboratif :

- **Tolérance aux pannes et aux partitions réseau** : les CRDT permettent aux utilisateurs de continuer à modifier le document même en cas de déconnexion temporaire, en assurant une réconciliation automatique lors de la reconnexion.
- **Réduction de la complexité serveur** : contrairement à l'Operational Transformation (OT), aucun serveur central n'est requis pour transformer ou ordonnancer les opérations, ce qui simplifie l'architecture et améliore la scalabilité horizontale.
- **Expérience utilisateur fluide** : les modifications locales sont appliquées immédiatement, sans attendre la validation d'un serveur, offrant ainsi une interface réactive et temps réel.
- **Extensibilité et maintenabilité** : les CRDT se prêtent facilement à l'intégration de nouvelles fonctionnalités collaboratives, telles que la gestion de versions, l'édition multi-documents et les plugins personnalisés, tout en préservant la convergence du système.
- **Sécurité et résilience** : la décentralisation des opérations réduit les risques de points de défaillance uniques et limite l'exposition aux interruptions de service ou aux attaques ciblant un serveur central.

En somme, le choix des CRDT s'inscrit parfaitement dans la logique d'un IDE collaboratif distribué : il garantit la robustesse, la cohérence automatique et la fluidité de l'édition multi-utilisateurs, tout en simplifiant la conception et en assurant une grande flexibilité pour des évolutions futures. Ce compromis entre performance, fiabilité et simplicité d'implémentation rend cette technologie particulièrement adaptée aux besoins de collaboration en temps réel et aux exigences d'un environnement distribué hétérogène.

Chapitre 3

État de l'Art des IDE Modernes

3.1 Analyse comparative

Les environnements de développement modernes tels que Visual Studio Code, IntelliJ IDEA ou Eclipse proposent des fonctionnalités avancées : gestion de projets, coloration syntaxique intelligente, débogage intégré, extensions modulaires et collaboration en temps réel.

Les solutions collaboratives existantes s'appuient souvent sur des architectures client-serveur centralisées et des mécanismes de synchronisation propriétaires.

3.2 Limites des solutions existantes

Malgré leurs performances, ces environnements présentent certaines limites :
- Dépendance à des serveurs cloud propriétaires.
- Difficulté d'intégration dans des environnements locaux distribués.
- Manque de contrôle sur les mécanismes internes de synchronisation.

Ces limites motivent la conception d'un éditeur collaboratif basé sur des mécanismes ouverts et optimisés, adaptés aux environnements distribués hétérogènes.

Deuxième partie

Conception et Architecture du Système

Chapitre 1

Analyse et Spécification des Besoins

L'analyse des besoins constitue une phase structurante dans la conception d'un environnement de développement intégré collaboratif distribué. Elle permet de formaliser de manière rigoureuse les exigences fonctionnelles et non fonctionnelles du système, tout en intégrant les contraintes techniques et technologiques inhérentes au contexte de développement. Cette étape conditionne la cohérence architecturale globale ainsi que la robustesse des choix technologiques retenus.

1.1 Exigences fonctionnelles

Le système doit permettre une **édition collaborative de code en temps réel**, reposant sur un mécanisme de réplication cohérente basé sur les **CRDT (Conflict-free Replicated Data Types)** afin de garantir la convergence sans mécanisme central de résolution de conflits.

Il doit intégrer un **gestionnaire de projets** compatible avec les structures standards telles que **Maven** et **Gradle**, incluant la détection automatique des dépendances et la configuration d'exécution.

L'éditeur doit proposer des fonctionnalités avancées : **coloration syntaxique multi-langages**, **analyse lexicale incrémentale**, **détection d'erreurs en temps réel**, **complétion intelligente via LSP**, **navigation sémantique dans le code**.

Le système doit supporter une **authentification multi-utilisateurs**, la gestion de **sessions collaboratives isolées**, ainsi qu'une synchronisation bidirectionnelle basée sur le protocole **WebSocket**.

Enfin, un **système de plugins extensible** doit permettre l'ajout dynamique de nouvelles fonctionnalités sans modification du noyau applicatif, garantissant ainsi l'évolutivité du système.

1.2 Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes et qualités que le système doit respecter, au-delà des fonctionnalités strictes. Elles concernent la performance, la sécurité, la fiabilité, la scalabilité et la maintenabilité du système.

Tout d'abord, le système doit garantir une **latence de synchronisation inférieure à 100 ms** dans un environnement réseau standard. Cette contrainte est essentielle pour assurer une expérience utilisateur fluide et réactive, notamment lors de l'édition collaborative en temps réel sur des documents distribués.

Ensuite, le système doit assurer une **cohérence éventuelle forte** entre toutes les réplicas distribuées. Chaque modification doit converger vers un état global cohérent, indépendamment de l'ordre de réception des opérations et de leur propagation dans le réseau. Cette propriété garantit l'intégrité des données et la continuité de l'expérience collaborative.

L'architecture doit également permettre une **scalabilité horizontale**, c'est-à-dire la capacité d'ajouter dynamiquement de nouveaux nœuds serveurs afin de traiter des charges de travail croissantes sans dégradation des performances. Cette scalabilité doit être supportée par des mécanismes de conteneurisation et d'orchestration distribuée, assurant la flexibilité et la robustesse du déploiement.

La **sécurité** constitue un autre aspect fondamental des exigences non fonctionnelles. Le système doit protéger les données et les sessions utilisateur grâce à plusieurs mesures complémentaires : un **chiffrement des communications** via TLS, garantissant la confidentialité des informations transmises ; une **isolation stricte des sessions** pour éviter les interférences entre utilisateurs ; un **contrôle d'accès basé sur rôles (RBAC)** pour limiter les privilèges selon le profil utilisateur ; et une **validation systématique des opérations reçues** afin de prévenir toute modification non autorisée ou malformée.

Enfin, le système doit respecter les principes de **modularité, maintenabilité et testabilité**, permettant une évolution incrémentale et un déploiement fiable sur différentes plateformes. La **portabilité multiplateforme** sur Windows, Linux et macOS constitue une exigence essentielle pour garantir l'accessibilité et l'utilisation du système dans des environnements variés.

En résumé, ces exigences non fonctionnelles définissent un cadre strict pour la conception et le développement du système, garantissant des performances optimales, une sécurité renforcée, une robustesse face aux pannes et une évolutivité adaptée aux besoins des environnements distribués collaboratifs.

1.3 Contraintes

La conception et le développement du système doivent respecter un ensemble de contraintes techniques et environnementales spécifiques, qui conditionnent la faisabilité et la performance globale de l'éditeur collaboratif.

Tout d'abord, le développement est réalisé en **Java 21 ou supérieur**, afin de bénéficier des dernières améliorations de la plateforme, notamment en termes de performance, de gestion de la mémoire et de sécurité. Cette contrainte impose également la compatibilité avec les bibliothèques et frameworks modernes, ainsi qu'une maintenance facilitée sur le long terme.

L'interface utilisateur est implémentée en **JavaFX**, choix motivé par la possibilité de concevoir des interfaces graphiques riches, réactives et multiplateformes, tout en assurant une intégration harmonieuse avec le moteur applicatif Java. Cette contrainte oriente également le design de l'interface vers une architecture basée sur le pattern MVVM ou MVC, garantissant modularité et testabilité.

La communication entre les différents clients et le serveur repose sur le protocole **WebSocket**, permettant un échange bidirectionnel et en temps réel des opérations collaboratives. Cette contrainte impose une attention particulière à la gestion des connexions, à la tolérance aux pannes et à la sécurisation des flux de données.

Enfin, le système doit être pleinement compatible avec les principaux systèmes d'exploitation (**Windows, Linux et macOS**), garantissant ainsi une portabilité multiplateforme et une accessibilité maximale pour les utilisateurs dans des environnements hétérogènes. Cette contrainte influe sur le choix des dépendances, la configuration des chemins d'exécution et la gestion des différences spécifiques à chaque OS.

En résumé, ces contraintes définissent le cadre technique et opérationnel du projet, assurant la cohérence entre les choix technologiques, la performance attendue et l'expérience utilisateur finale.

Chapitre 2

Architecture Générale du Système

L'architecture adoptée repose sur une approche **modulaire orientée services**, combinant une structure client-serveur temps réel et un noyau applicatif extensible. Cette organisation vise à séparer strictement les responsabilités afin de garantir robustesse et évolutivité.

2.1 Architecture logicielle modulaire

Le système est structuré selon une architecture en couches clairement définies :

Couche Présentation : interface JavaFX, composants UI réactifs, gestion des interactions utilisateur.

Couche Application : orchestration des services, gestion des commandes, coordination des opérations collaboratives.

Couche Domaine : implémentation du moteur CRDT, gestion de l'état partagé, validation des opérations.

Couche Infrastructure : communication WebSocket, persistance, sécurité, gestion des connexions.

Cette séparation favorise l'**indépendance des modules**, la **testabilité unitaire** et l'**évolution incrémentale** du système.

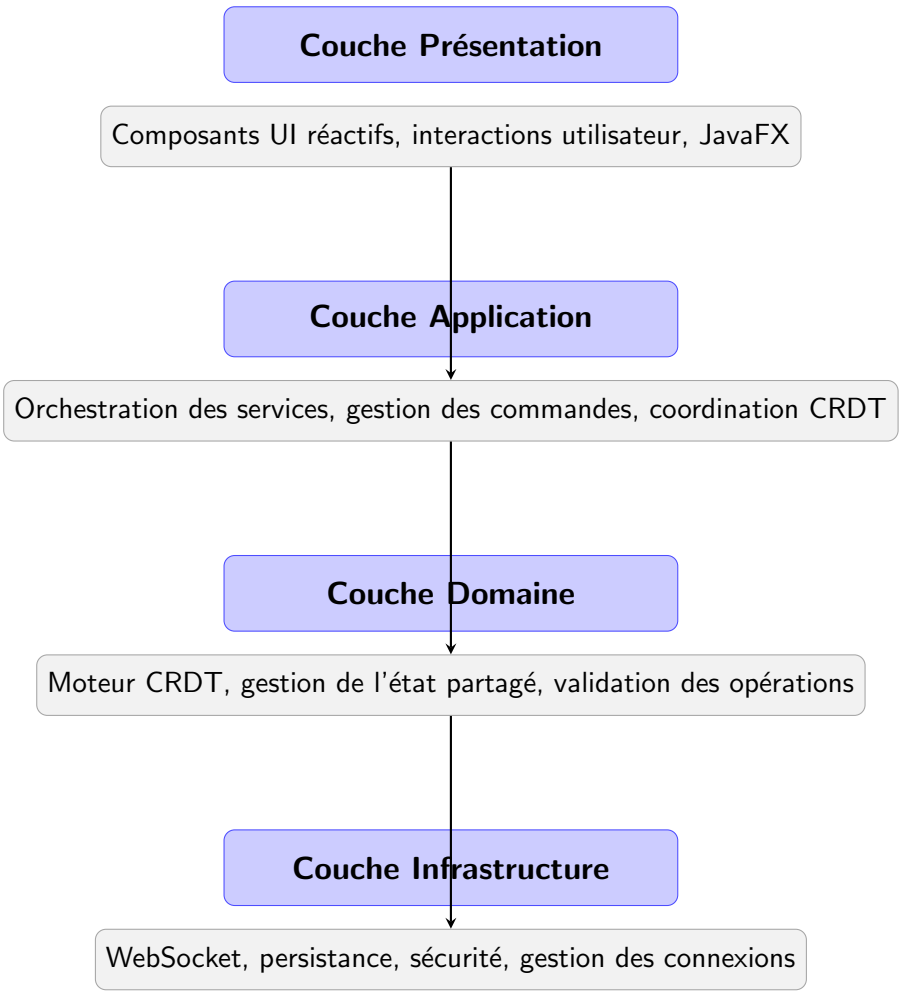


Figure 2.1 – Architecture logicielle modulaire de l'IDE collaboratif avec ses couches et modules principaux

2.2 Architecture réseau

Le système adopte un modèle **client-serveur bidirectionnel** basé sur WebSocket.

Chaque client maintient une copie locale du document sous forme de structure CRDT. Les opérations locales sont propagées sous forme d'**opérations immuables** vers le serveur, qui agit principalement comme **relai de diffusion** sans transformation centrale, garantissant ainsi une convergence déterministe.

Un mécanisme de **gestion de sessions collaboratives** assure l'isolation logique des espaces de travail et la synchronisation d'état lors de la reconnexion d'un client.

2.3 Modélisation UML

La modélisation du système repose sur :

Diagrammes de cas d'utilisation pour formaliser les interactions utilisateur.

Diagrammes de classes pour structurer le noyau CRDT, les gestionnaires de sessions et les composants applicatifs.

Diagrammes de séquence pour modéliser le flux des opérations collaboratives.

Diagrammes de déploiement pour représenter l'architecture distribuée.

Chapitre 3

Conception Technique Détaillée

3.1 Moteur CRDT

Le moteur repose sur une structure de type **CRDT séquentiel**, inspirée des modèles **RGA** ou **LSEQ**.

Chaque insertion est identifiée par un identifiant unique structuré sous la forme :

$$(\mathsf{siteID}, \mathsf{logicalClock})$$

La convergence est assurée par construction, sans coordination centrale ni verrou global.

Un mécanisme de **garbage collection logique** est envisagé afin de limiter la croissance structurelle des identifiants et optimiser l'empreinte mémoire.

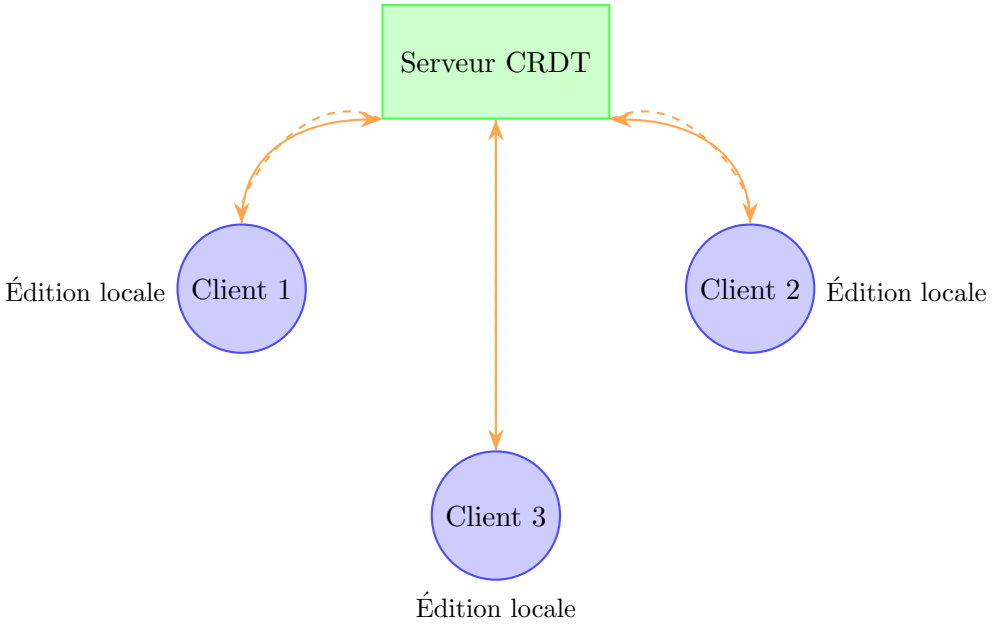


Figure 3.1 – Synchronisation et diffusion des opérations CRDT

3.2 Serveur WebSocket

Le serveur WebSocket constitue le cœur de la synchronisation en temps réel dans l'éditeur collaboratif. Il adopte un modèle **non bloquant basé sur NIO (Non-blocking I/O)**, permettant de gérer simultanément de nombreux clients sans surcharger les ressources. Cette approche assure une latence minimale et une réactivité optimale, même en présence de multiples utilisateurs connectés.

Le serveur a plusieurs responsabilités principales : **Distribution des opérations** : chaque modification effectuée par un utilisateur est transformée en opération immuable et diffusée aux autres clients participants de manière quasi instantanée.

Gestion des salles collaboratives : les documents ou projets sont isolés dans des sessions distinctes afin de garantir la confidentialité et l'intégrité des données.

Synchronisation initiale d'état : lors de la connexion d'un nouveau client, le serveur transmet l'état courant du document afin de permettre une collaboration fluide dès le départ.

Persistance optionnelle de snapshots : le serveur peut enregistrer périodiquement l'état des documents pour permettre la récupération après panne ou déconnexion prolongée.

Une stratégie de **déploiement par conteneurisation Docker** assure la **scalabilité horizontale**, facilitant l'ajout de serveurs supplémentaires pour gérer l'augmentation du nombre d'utilisateurs sans dégrader les performances.

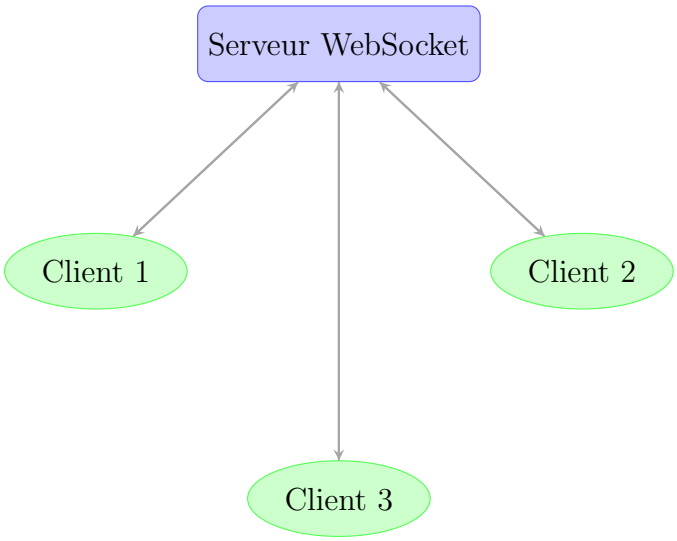


Figure 3.2 – Distribution des opérations

3.3 Interface JavaFX

L'interface utilisateur est conçue selon le modèle architectural **MVVM (Model-View-ViewModel)**, permettant une séparation stricte entre la logique métier, la représentation visuelle et la gestion des états. Cette approche favorise la **maintenabilité**, la **testabilité** et la possibilité d'évolution incrémentale de l'interface.

Les composants graphiques sont connectés aux modèles de données via des **propriétés observables** et des **bindings**, garantissant une réactivité **temps réel** lors des modifications de l'état du système. Cette liaison dynamique assure que les changements effectués par le moteur CRDT ou par l'utilisateur sont immédiatement reflétés dans l'interface, sans nécessiter de rafraîchissement manuel.

Un moteur de **coloration syntaxique incrémentale** est intégré afin de gérer efficacement les modifications partielles du document. Contrairement aux méthodes classiques qui recalculent entièrement le rendu après chaque modification, ce moteur analyse uniquement les portions affectées, réduisant ainsi la latence et optimisant les performances de l'application, même en présence de documents volumineux.

L'interface inclut également des mécanismes visuels d'alerte et de notification, permettant d'informer l'utilisateur de la synchronisation des opérations, de la détection de conflits ou des changements réalisés par d'autres collaborateurs, contribuant ainsi à une expérience utilisateur fluide et intuitive.

3.4 Gestionnaire de projets

Le gestionnaire de projets constitue le cœur de l'organisation et de la manipulation des projets au sein de l'application. Il offre une interface unifiée pour la **création de nouveaux projets** ainsi que pour l'**importation de projets existants**, garantissant une intégration fluide dans l'environnement de travail.

Chaque projet est soumis à un processus d'**indexation et d'analyse des fichiers**, permettant d'établir une cartographie complète des sources, des dépendances et des ressources associées. Cette étape est essentielle pour optimiser les fonctionnalités d'édition, de recherche et de navigation dans le code.

Le gestionnaire assure également une **intégration transparente avec les outils de build** tels que **Maven**, facilitant la compilation, la gestion des dépendances et l'exécution des tests unitaires. Cette intégration automatise la configuration du projet et réduit les interventions manuelles de l'utilisateur.

Enfin, le système prend en charge le **Language Server Protocol (LSP)** pour l'analyse statique, offrant des fonctionnalités avancées telles que l'autocomplétion, la vérification de types, la détection d'erreurs et la navigation intelligente dans le code, contribuant ainsi à un environnement de développement robuste, réactif et ergonomique.

3.5 Système de plugins

Le système de plugins constitue un mécanisme central pour l'**extensibilité et la modularité** de l'application. Il repose sur un **chargement dynamique des modules** via le service Java **ServiceLoader**, permettant d'ajouter ou de retirer des fonctionnalités sans modifier le noyau applicatif.

Chaque plugin doit implémenter une **interface contractuelle standardisée**, garantissant une intégration sécurisée et cohérente avec le reste du système. Cette approche assure que l'extension des fonctionnalités ne compromet pas la stabilité du cœur applicatif et maintient une **séparation stricte des responsabilités** entre les modules.

L'architecture ainsi proposée favorise :

- La **pérennité du système**, en facilitant la maintenance et les mises à jour incrémentales.
- L'**interopérabilité**, en permettant à des développeurs tiers de créer des extensions compatibles.
- L'**adaptabilité**, en autorisant l'activation ou la désactivation dynamique de fonctionnalités selon les besoins de l'utilisateur ou du projet.

Dans un contexte scientifique ou industriel, ce modèle garantit une **évolution contrôlée du logiciel**, sans répercussions sur les composants critiques et en conservant un haut niveau de fiabilité.

Troisième partie

Implémentation et Validation

Chapitre 4

Implémentation

4.1 Technologies utilisées

4.2 Structure du projet

4.3 Extraits de code

```
public void applyOperation(Operation op) {
        // Implementation
}
```

Chapitre 5

Tests et Validation

5.1 Tests unitaires

5.2 Tests d'intégration

5.3 Tests multi-clients

5.4 Tests de performance

Chapitre 6

Résultats et Analyse

6.1 Performance

6.2 Robustesse

6.3 Comparaison avec IDE existants

Chapitre 7

Coût et Perspectives

7.1 Estimation du coût

7.2 Limites

7.3 Améliorations futures

Chapitre 8

Conclusion Générale

Ce projet démontre que ...

Références bibliographiques et webographiques

Annexe

Questions d'enquête :

4. Pensez-vous que l'utilisation des éléments multimédias rend les cours de SVT plus intéressants ?
-Oui, beaucoup plus intéressants
-Oui, un peu plus intéressants
-Non, cela ne change rien
-Non, cela rend le cours moins intéressant

5. Quel est, selon vous, l'effet principal des supports multimédias sur votre compréhension des notions de SVT ?
-Amélioration significative de la compréhension
-Légère amélioration
-Pas d'effet notable
-Cela complique parfois la compréhension

6. Par rapport aux cours traditionnels (exposés magistraux, lecture de manuels), les supports multimédias vous aident-ils à mieux mémoriser les notions ?
-Oui, beaucoup mieux
-Oui, un peu mieux
-Non, aucun changement
-Non, cela me perturbe

7. Quels sont les avantages des supports multimédias en SVT selon vous ?
-Visualisation concrète des phénomènes biologiques et géologiques
-Meilleure motivation et engagement en classe
-Apprentissage plus rapide et efficace
-Interactivité et participation active
-Autres....

8. Avez-vous rencontré des difficultés lors de l'utilisation des supports multimédias en cours de SVT ?
-Oui, problèmes techniques (connexion, matériel, compatibilité)
-Oui, surcharge d'informations qui rend l'apprentissage confus
-Non, aucune difficulté

9. Préférez-vous apprendre avec des supports multimédias ou avec des supports traditionnels (manuels, notes de cours) ?
-Uniquement avec des supports multimédias

-Un mélange des deux
-Uniquement avec des supports traditionnels

10. Selon vous, l'intégration des éléments multimédias devrait-elle être renforcée dans l'enseignement des SVT ?
-Oui, fortement
-Oui, mais avec modération
-Non, cela n'est pas nécessaire